

MediSync

Medication tracking for people who see more than one doctor

Darshita Bansal | B.Tech AI & DS, IIT Raichur | August 2026

1. What this is

MediSync is a phone app for tracking prescribed medicines. Two things make it different from a normal reminder app.

First, it remembers which doctor prescribed what. That lets it notice when two doctors have prescribed the same thing without knowing about each other.

Second, more than one person can use it for the same patient. The patient takes the medicine, but their children and whoever helps at home can all see the schedule and mark doses as taken.

This is the third version of the idea. The first was a plain reminder app built on the openFDA API, which didn't work out — openFDA has no usable interaction data and doesn't cover Indian brand names at all. The second was about overdoses from over-the-counter medicines, which turned out to be the wrong problem to solve.

2. The problem

Take a 68-year-old with BP and diabetes. She sees a cardiologist, an endocrinologist, and a local doctor. None of them sees the others' prescriptions. She is the only person in all three rooms, and she is the least likely to notice when something doesn't add up.

So things slip. Two doctors prescribe the same drug under different brand names. One doctor changes a dose and she keeps following the old one. A five-day antibiotic course runs into week three because nobody told her to stop. A specialist stops something and the tablet stays in the daily box anyway.

Doctors call this medication reconciliation. It is a known source of harm, especially after a hospital discharge.

There is a second problem. The person taking the medicine is often not the person managing it. A daughter in Bangalore orders the refills, a son in Pune tracks the appointments, and someone at home hands over the tablet every morning. Most medicine apps assume one person with one phone, which is wrong for exactly the people who need the app most.

And a reminder that fires and disappears doesn't prove anything. Without a confirmation, the app is recording what should have happened, not what did.

3. What it does

Reminders that don't give up

When a dose is due, the app sends a notification with a Taken button on it. Tapping that button records the dose without opening the app. If nobody taps anything, it reminds again after 15

minutes, then again after 30. If there is still no response at 45 minutes, it marks the dose as missed and notifies the family.

Three reminders is the limit, and Snooze is always there. Nagging people about medicine tends to backfire, and an app that won't shut up gets uninstalled.

The timers also run on the server, not just on the phone. If the phone is switched off or the app has been killed, the family still gets told.

Catching conflicts between doctors

Every medicine entry stores which doctor prescribed it, when, and for how long. Whenever something new is added, the app checks it against everything currently active and flags problems in four levels:

Level	What it catches	What data it needs
1	The same brand prescribed by two doctors, contradicting instructions, courses that should have ended	Nothing
2	The same ingredient under different brand names	Brand to ingredient list
3	Two different drugs from the same class	Ingredient to drug class list
4	Combinations known to be risky together	A hand-written rule list

Level 1 needs no outside data at all, so it works from day one. Each level after that is a bonus, not a dependency. This matters because the drug data is the riskiest part of the project, and this way it can't hold up the build.

When something is flagged, the app says what it found and who prescribed what. It does not tell anyone to stop taking anything. For example: "Dr. Sharma prescribed Telma 40 on 12 Aug. Dr. Rao prescribed Telmiking 40 on 3 Sep. Both contain telmisartan. Please check with either doctor." Never "you are double-dosing, stop one."

Sharing with family

Each patient has an account, and they can invite other people to it. Four levels of access:

Access	Can do
Owner	Everything, including inviting and removing people
Family admin	See everything, edit medicines, get missed-dose alerts
Helper	See today's schedule and mark doses as taken
Viewer	Read only

Nobody gets added without being invited and accepting. The patient can see who has access and remove anyone instantly. Every action gets logged, because this is health data being shared between people and you want a record of who did what.

One case worth thinking about: if the patient has memory problems, "the patient controls access" gets complicated. For now the patient's account is the owner wherever possible, and a family member owning it is a deliberate choice that gets recorded rather than a quiet default.

Working offline

Phones lose signal, so the app saves everything locally and syncs when it reconnects. The interesting case is two people marking the same dose as taken — the helper offline at 8:03, the daughter online at 8:04. That's normal, not an error. Whichever reaches the server first becomes the record, and the second person just sees "already confirmed by Priya at 8:04" instead of an error or a duplicate.

Looking up medicines

Indian medicines are sold by brand name, and openFDA doesn't know any of them. But it does know ingredient names. So the app converts "Telma 40" into "telmisartan 40mg" first, then looks that up to show what the medicine is for and what the side effects are.

The conversion handles typos and spacing with fuzzy matching. If it isn't confident, it shows a few options and asks rather than guessing. Medicines it can't identify still get reminders normally, they just sit out of the conflict checking, and the app says so.

4. How it's built

A Flutter app on the phone, a Node and Express server, and PostgreSQL behind it. A background worker generates the upcoming doses and runs the escalation timers. Redis holds the timers and pushes live updates. WebSocket keeps everyone's phone in sync, so the daughter in Bangalore sees the morning dose confirmed the moment the helper taps it.

Part	Using	Why
Phone app	Flutter	One codebase for Android and iOS, and its notification library supports buttons
Server	Node.js + Express	Already what the repo is set up with
Database	PostgreSQL	Needs real relationships and date handling, and pg_trgm does the fuzzy matching in SQL
Live updates	WebSocket	Polling would be wasteful for this
Timers and cache	Redis	Escalation timers that survive the phone being off
Background jobs	node-cron	Generating doses and checking for missed ones
Login	JWT	Same account on several devices
Hosting	Render or Railway, Neon for the database	Free tier, and having a live link matters

Things that will actually be annoying

Android 12 and above needs a special permission for exact alarms, and Doze mode kills the inexact ones. Xiaomi, Oppo, Vivo and Realme phones are worse than stock Android about this and will need testing on real devices.

iOS only allows 64 pending notifications at a time. Four medicines twice a day uses that up in about a week, so the app has to top up its schedule every time it opens.

5. Database

The important part is that prescriptions are stored with a start date, an end date, and a link to whatever they replaced. That's what makes it possible to ask "what is she actually supposed to be taking right now" when courses overlap and get changed.

The other thing worth noting is that the schedule and the actual doses are separate tables. A single "was it taken" flag that gets overwritten every day makes adherence history impossible to reconstruct.

Table	Holds
users	login details
patients	who the medicines belong to, their timezone
patient_access	who can see this patient, at what level, who invited them
audit_log	every action taken, by whom, when
prescribers	the doctors
prescriptions	a visit — doctor, date, optional photo
prescription_items	one medicine: dose, frequency, start and end date, what it replaced
schedules	the repeating rule — 8am and 8pm, daily
dose_events	one row per actual dose, with the time it was confirmed and by whom
conflicts	what was flagged, which records were involved, whether it's been sorted out
brands / ingredients	brand name to ingredient mapping and drug classes

6. How the main parts work

Adding a prescription

```
Dr. Rao, 3 Sep, "Telmikind 40, once daily, 30 days"
-> Telmikind 40 becomes telmisartan 40mg
-> check against everything currently active
    telmisartan already there, from Dr. Sharma on 12 Aug
-> save the conflict, note both prescriptions
-> push it to everyone's phone
```

A missed dose

```
8:00    notification, buttons: Taken / Snooze / Skip
8:15    no response, remind again
8:30    no response, remind again
8:45    mark as missed, tell the family
```

Two people, one dose

8:03 helper taps Taken, but has no signal
8:04 daughter marks it taken in the app, saved
8:11 helper's phone reconnects and sends its copy
 -> server already has it, keeps the first one
 -> helper sees "already confirmed by Priya at 8:04"

7. What it doesn't do

This is deliberate, not missing:

- It never tells anyone to change or stop a medicine. It shows what it found and says to ask a doctor.
- No symptom checking or guessing what's wrong with you.
- It doesn't claim to catch every dangerous drug combination. Spotting the same drug twice is not the same as knowing every interaction, and the app has to be honest about that.
- It doesn't replace what a doctor or pharmacist does. It just makes sure they have the full list.
- No doctor login, and no doctor ratings — there is no legal way to get review data for that anyway.

8. Build order

Step	What
1	Server talking to the database, migrations set up
2	Login, multiple users per patient, access levels
3	Prescriptions with doctor, dates and duration
4	Level 1 conflict checking
5	Schedules and dose records
6	Flutter app — see medicines, mark doses
7	Taken button on the notification itself
8	Reminder escalation and family alerts
9	Live sync between phones
10	Offline queue and the duplicate-dose case
11	Brand to ingredient conversion, levels 2 and 3
12	Deploy it, write the README, set up CI

Step 8 is the finish line for a working version. Everything up to there is a complete app that can be demonstrated. Steps 9 to 11 add depth. If time runs out, a solid step 8 beats a half-finished step 11.

9. Later, if it goes well

- **Photograph the prescription** instead of typing it in. Best demo value by far. Printed and pharmacy-printed slips first, since handwriting defeats most OCR.
- **A chatbot that only answers about your own medicines** — can I take this on an empty stomach, I missed yesterday, what is this even for. Grounded in the patient's actual list, not general medical advice.
- **Suggesting which specialist to see** based on the drug classes someone is already on.
- **Refill warnings** from stock count and how regularly doses are actually taken.
- **BP and sugar logging**, with a printable summary for the next appointment.
- **ABDM**, India's national health data standard, apparently has a developer sandbox. Worth checking — building against a real standard would count for a lot.

MediSync is an information tool. It is not a medical device and does not give medical advice.